

TEMA 06 • ESTRATEGIA DEL DATO, ML & DL

Implementación y *despliegue* de modelos.

Llevar el modelo del cuaderno Jupyter al mundo real. Donde los usuarios, los datos vivos y las catástrofes ocurren.

ASIGNATURA	Estrategia del Dato, ML y DL
TEMA	06 • Implementación y Despliegue
DURACIÓN	120 minutos
SESIÓN	06 / 07

Dos bloques. Una clase. Del modelo al *servicio en producción*.

BLOQUE I · 60 MIN · DESPLEGAR

- Entornos: *dev · test · prod*
- MLOps: la metodología que lo une todo
- Ciclo de vida CI/CD/CT
- Servicios en la nube: AWS, Azure, GCP
- Contenedores: Docker y Kubernetes
- Caso real: *Netflix* en producción

BLOQUE II · 60 MIN · MANTENER VIVO

- Monitoreo de modelos en producción
- Data drift: *cuando el mundo cambia*
- Herramientas: MLflow, Kubeflow, Model Monitor
- Reentrenamiento: fijo vs. dinámico
- Roles y responsabilidades
- Quiz: ¿desplegarías esto?

Salimos del laboratorio. *Vamos a producción.*

En las sesiones 4 y 5 limpiamos datos y entrenamos modelos. Tenemos algo que funciona en nuestro portátil. Hoy lo convertimos en un servicio que millones de personas pueden usar — y que sigue funcionando mañana.



Aquí es donde mueren la mayoría de proyectos de IA. «*Funciona en mi máquina*» no es producción.

El **87%** de los modelos de ML nunca llegan a producción.

Entrenar un modelo no es el final. Es apenas la mitad del camino. Los que llegan a producción afrontan retos distintos:

- Datos que cambian con el tiempo
- Latencias y picos de tráfico
- Versiones del modelo conviviendo
- Equipos distintos: ciencia y operaciones

Producción = consistencia + escala + supervisión. Todo lo demás es prototipo.

ANALOGÍA · LA PAELLA PERFECTA

Cocinar una paella increíble en tu cocina no es lo mismo que servirla todos los domingos en un restaurante para 200 personas.

El plato tiene que ser **idéntico** cada vez, **escalar** en eventos grandes y los **ingredientes** tienen que llegar frescos cada mañana.

Desplegar.

Sacar el modelo del cuaderno. Llevarlo a un servidor.
Hacerlo accesible, reproducible y escalable.

Tres *entornos*. Tres propósitos.

Todo software vive en al menos tres entornos distintos. Cada uno tiene reglas, datos y usuarios propios.

DESARROLLO · DEV

Donde se construye

Jupyter Notebook, VSCode, datos de prueba, todo es modificable.

Cocina del chef: el plato se inventa, se rompe, se rehace mil veces.

PRUEBAS · TEST · STAGING

Donde se valida

Réplica de producción con datos sintéticos. QA, tests de integración, depuración.

Servicio de prueba antes de la inauguración del restaurante.

PRODUCCIÓN · PROD

Donde se sirve

Usuarios reales, datos reales, dinero real. Cero margen para fallos.

Restaurante abierto: si algo falla, los clientes se enteran.

REGLA DE ORO

Nunca pruebes en producción. Nunca despliegues sin haber pasado por TEST. *«Funciona en mi máquina»* es la frase más cara del software.

Lo que *nadie te cuenta* del despliegue.

Llevar un modelo a producción no es copiar el `.pkl` a un servidor. Hay cuatro frentes simultáneos.

01 · INTEGRACIÓN

Con sistemas existentes

APIs, bases de datos, microservicios. Tu modelo es un eslabón más.

02 · DEPENDENCIAS

Versiones que cambian

Python 3.9, sklearn 1.4, numpy 1.26... si una cambia, el modelo puede no funcionar igual.

03 · REPRODUCIBILIDAD

Mismo input, mismo output

Crítico en sanidad o finanzas: *tienes que poder explicar* cada predicción del pasado.

04 · DATOS VIVOS

Entradas que cambian

Lo que entra al modelo hoy no es lo que entraba el mes pasado. Hay que rastrearlo.

MLOps: cuando ML conoce a DevOps.

MLOps es la metodología que automatiza y estandariza el **ciclo de vida completo** de un modelo de machine learning — desde el desarrollo hasta el despliegue y la monitorización en producción.

- Combina *machine learning* + *ingeniería de datos* + *DevOps*
- Estandariza la entrega de nuevos modelos
- Gestiona código, datos **y** modelos como un único producto
- Reduce el «funciona en mi máquina» a cero

MACHINE LEARNING
■ Modelos, experimentos, métricas

DATA ENGINEERING
■ Pipelines, calidad, versionado

DEVOPS
■ CI/CD, despliegue, monitorización

= *MLOps*
LA INTERSECCIÓN OPERATIVA

De *cero* a *full auto*.

NIVEL 0 · MANUAL

Todo a mano

Notebook → script → ZIP por email → operaciones lo despliega.

- Despliegue trimestral o anual
- Cero CI/CD
- Sin monitorización activa
- Re-entreno: por crisis

NIVEL 1 · PIPELINE AUTOMATIZADO

El entrenamiento se automatiza

Pipelines reproducibles. Trigger automático ante nuevos datos.

- CI/CD del modelo
- Monitoring básico
- Re-entreno automático con datos frescos
- Validación previa al deploy

NIVEL 2 · CI/CD/CT COMPLETO

Todo está automatizado

Cambios de código y de datos disparan despliegues seguros.

- Continuous Integration
- Continuous Delivery
- Continuous Training
- Equipos de plataforma dedicados

La mayoría de equipos están en nivel 0 o 1. Llegar al 2 es *una decisión estratégica*, no solo técnica.

Las *tres ruedas* que nunca paran de girar.

En el software clásico hay CI y CD. En ML añadimos CT — porque el modelo, a diferencia del código, **se degrada solo** con el tiempo.

CI · CONTINUOUS INTEGRATION

Integración continua

Cada cambio de código se valida automáticamente: tests unitarios, validación de datos, validación del modelo.

Símil cine: cada vez que un actor cambia una línea, hay un ensayo express con todo el equipo.

CD · CONTINUOUS DELIVERY

Entrega continua

Cada modelo validado se prepara para desplegarse en producción. Despliegues seguros con rollback.

Símil: el catering ya está envasado y etiquetado. Solo falta servir.

CT · CONTINUOUS TRAINING

Reentrenamiento continuo

El modelo se reentrena automáticamente cuando llegan nuevos datos o se detecta degradación.

Específico de ML: el código no caduca; el modelo sí.

Cuatro *roles*, una cocina.



DISEÑA EL PLATO

Data Scientist

Explora datos, prueba algoritmos, entrena modelos en notebooks.



TRAE LOS INGREDIENTES

Data Engineer

Construye pipelines de datos, ETL, garantiza calidad y disponibilidad.



MONTA LA COCINA INDUSTRIAL

ML Engineer

Empaqueta el modelo, lo despliega, automatiza pipelines, MLOps.



MANTIENE EL RESTAURANTE ABIERTO

DevOps / SRE

Infraestructura, escalado, alertas, fiabilidad operativa 24/7.

En equipos pequeños, una persona hace varios roles. En empresas grandes, son departamentos enteros que tienen que *hablar entre sí*. MLOps es, sobre todo, una forma de organizar esa conversación.

La *nube*.

No tienes que comprarte un servidor. Alquilas el que necesitas, cuando lo necesitas, y pagas por minuto.

Por qué *casi todo el ML* vive en la nube.

- **Flexibilidad:** empiezas con 1 GPU, escalas a 1.000 sin cambiar de empresa
- **Coste:** pagas por uso, no por capacidad ociosa
- **Agilidad:** despliegas un modelo en minutos, no en semanas
- **Seguridad y compliance:** certificaciones que tu empresa no haría sola
- **Acceso remoto:** equipos distribuidos colaborando en el mismo proyecto

ANALOGÍA · CASA PROPIA VS HOTEL

Montar tu propia infraestructura es **comprarte una casa**: hipoteca, mantenimiento, reformas, vecinos. Compromiso a largo plazo.

La nube es un **hotel**: pides la habitación que necesitas, pagas por noches y, cuando sobra, te vas. *No hace falta tener clavos en casa por si se rompe una silla.*

Tres proveedores dominan el mercado: **AWS** (Amazon), **Azure** (Microsoft) y **GCP** (Google). Más del 65 % de la cuota global entre los tres.

Los *tres reinos* del cloud ML.

Las tres plataformas hacen **lo mismo** con nombres distintos. La elección depende del ecosistema en el que ya esté tu empresa.

	AWS AMAZON	AZURE MICROSOFT	GCP GOOGLE
Plataforma ML	SageMaker — IDE Jupyter integrado, AutoML, deploy.	Azure ML — pipelines, MLOps integrado, IA responsable.	Vertex AI — AutoML, Model Garden, IA generativa.
Sin código	SageMaker Canvas	Designer (drag & drop)	Vertex AI AutoML
Monitorización	SageMaker Model Monitor	Azure ML + Event Grid	Vertex AI Model Monitoring
Fortaleza típica	Cuota de mercado, ecosistema enorme.	Empresas con Microsoft 365 / Active Directory.	Equipos data-first, IA generativa Gemini.

Caso *Netflix*: 260 millones de cuentas, un modelo a la vez.

LO QUE TIENEN MONTADO

- **AWS** como infraestructura base desde 2008
- *Metaflow*: su framework MLOps interno (hoy open-source)
- Miles de modelos en producción a la vez: recomendaciones, miniaturas, codificación de vídeo, fraude...
- Cada usuario ve una *portada distinta* generada por modelos
- Re-entrenamientos continuos según hábitos de visionado

LAS CLAVES DEL ÉXITO

01 · PLATAFORMA COMÚN

Cualquier científico de datos puede pasar de notebook a producción con una sola herramienta.

02 · A/B TESTING BRUTAL

Todo modelo nuevo se compara contra el actual con tráfico real. Solo gana si mejora el engagement.

03 · MONITORIZACIÓN 24/7

Drift, latencia, errores. Alertas automáticas si algo se degrada.

Contenedores.

Una manera elegante de decir: «mi modelo viaja con todo lo que necesita para funcionar — sin sorpresas».

La *paella precocinada*.

Imagina que tu paella sale perfecta en tu cocina. Quieres que un amigo en Berlín la cocine exactamente igual.

Le envías: **la receta, el arroz exacto, el azafrán, la paellera del tamaño correcto, el fuego adecuado...** todo dentro de un mismo paquete sellado.

Tu amigo abre el paquete, calienta el fuego y la paella sale *idéntica* a la tuya. Sin discusiones sobre si su sartén es distinta o si su gas calienta más fuerte.

CONTENEDOR · PAELLA.TAR

 Arroz bomba — 400 g

 Azafrán — 0,2 g

 Marisco — 600 g

 Paellera — Ø 38 cm

 Receta — paella.recipe v2.1

 Tiempo — 25 min

Mismo paquete → misma paella
en cualquier cocina

TRADUCCIÓN AL ML

Un **contenedor** empaqueta tu modelo + Python + librerías + configuración. Ejecutas el contenedor en cualquier servidor y el modelo se comporta exactamente igual. *Sin «funciona en mi máquina».*

Docker: la herramienta de paella precocinada estándar.

Docker es la plataforma de contenedores más popular. Open source. Casi todo el ML en producción del mundo usa Docker.

- Empaqueta una aplicación con todas sus dependencias
- Se ejecuta igual en tu portátil, en AWS o en un servidor de la oficina
- *Docker Hub*: registro público de imágenes (como GitHub para contenedores)
- Aislamiento entre contenedores: si uno cae, los demás siguen

```
# Dockerfile – receta del contenedor
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY model.pkl .
COPY serve.py .

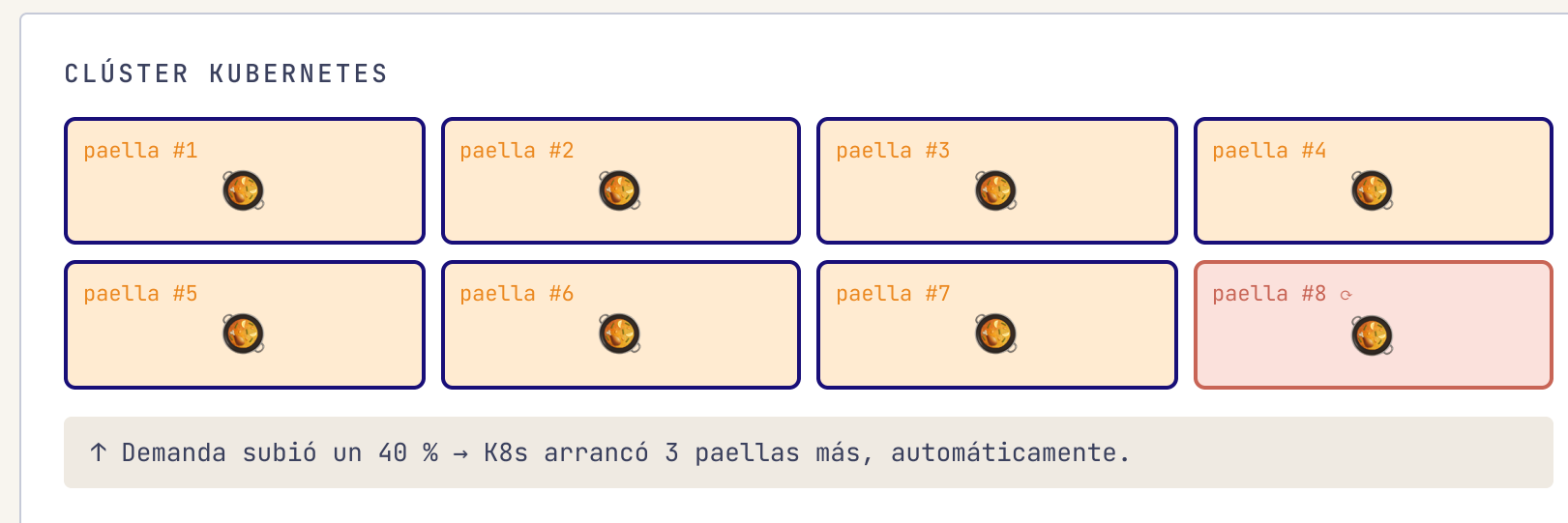
EXPOSE 8000
CMD ["python", "serve.py"]
```

Una imagen Docker es una receta paso a paso. Reproducible al 100 %.

Kubernetes: la cocina industrial.

Si Docker es *una paella precocinada*, Kubernetes es la **cocina industrial que sirve 5.000 paellas a la vez**, repone cuando se acaba el arroz y abre más fuegos cuando llega el rush.

- Orquesta cientos o miles de contenedores
- Escala automáticamente con la demanda
- Si un contenedor se cae, lo reinicia solo
- Reparte tráfico entre las réplicas
- Despliegues sin downtime (rolling updates)



Kubernetes (K8s) decide cuándo abrir fuegos nuevos, cuándo cerrarlos y qué hacer si uno se apaga.

Docker *empaqueta*. Kubernetes *orquesta*.

DOCKER

Una paella precocinada

Empaqueta tu aplicación con sus dependencias en un contenedor portable.

Pregunta que responde: «¿cómo hago que mi modelo funcione igual en cualquier servidor?»

- Construye imágenes
- Ejecuta contenedores individuales
- Una máquina, varios contenedores

KUBERNETES

La cocina industrial

Gestiona miles de contenedores a la vez. Escala, repone, balancea.

Pregunta que responde: «¿cómo manejo 10.000 paellas funcionando a la vez sin que se queme ninguna?»

- Orquesta clústeres de contenedores
- Auto-scaling, auto-healing
- Muchas máquinas, miles de contenedores

EN LA PRÁCTICA

Empaquetas tu modelo con *Docker*. Lo desplegas a escala con *Kubernetes*. Son **complementarios**, no alternativas.



INTERMEDIO • PERSONAJE

Margaret *Hamilton.*

Inventó el término «*software engineering*» y escribió el código que llevó al Apollo 11 a la Luna. Diseñó pensando en cómo iba a *fallar* el sistema — exactamente la mentalidad de MLOps, **50 años antes**.

1961

Entra al MIT con 24 años.
Única mujer del equipo.

1965

Lidera el software del
Apollo. Debe funcionar a la
primera.

1969

Su sistema evita un fallo
crítico en el aterrizaje
lunar.

1986

Funda Hamilton Tech.
Codifica lo que hoy es
DevOps / MLOps.

POR QUÉ LA TRAEMOS HOY

Graceful degradation, versionado, especificaciones formales, trazabilidad. Cada idea que damos por sentada en producción la empezó ella. **Producción es donde se ve qué decisiones de diseño fueron buenas.**

Mantener *vivo.*

Desplegar es el principio, no el final. Los modelos en producción se degradan solos. Monitorizarlos y reentrenarlos es la otra mitad del trabajo.

El modelo *en producción* es un ser vivo.

Una vez en producción, los modelos pueden degradarse con el tiempo debido a cambios en los patrones de datos subyacentes. Es vital hacer seguimiento continuo del rendimiento.

- Métricas de modelo en vivo: *accuracy, recall, F1*
- Métricas de sistema: latencia, throughput, uso de memoria
- Calidad de datos de entrada
- Alertas automáticas cuando algo se desvía
- Dashboards visuales para todo el equipo

SÍMIL · EL RESTAURANTE

Inaugurar el restaurante no es el final. Hay que revisar cada noche:

¿Sigue saliendo igual la paella? ¿La clientela ha cambiado? ¿El proveedor cambió el arroz? ¿Los nuevos camareros cocinan igual?

Si no miras, no te enteras hasta que llegan las reseñas malas. *Para entonces, ya has perdido clientes.*

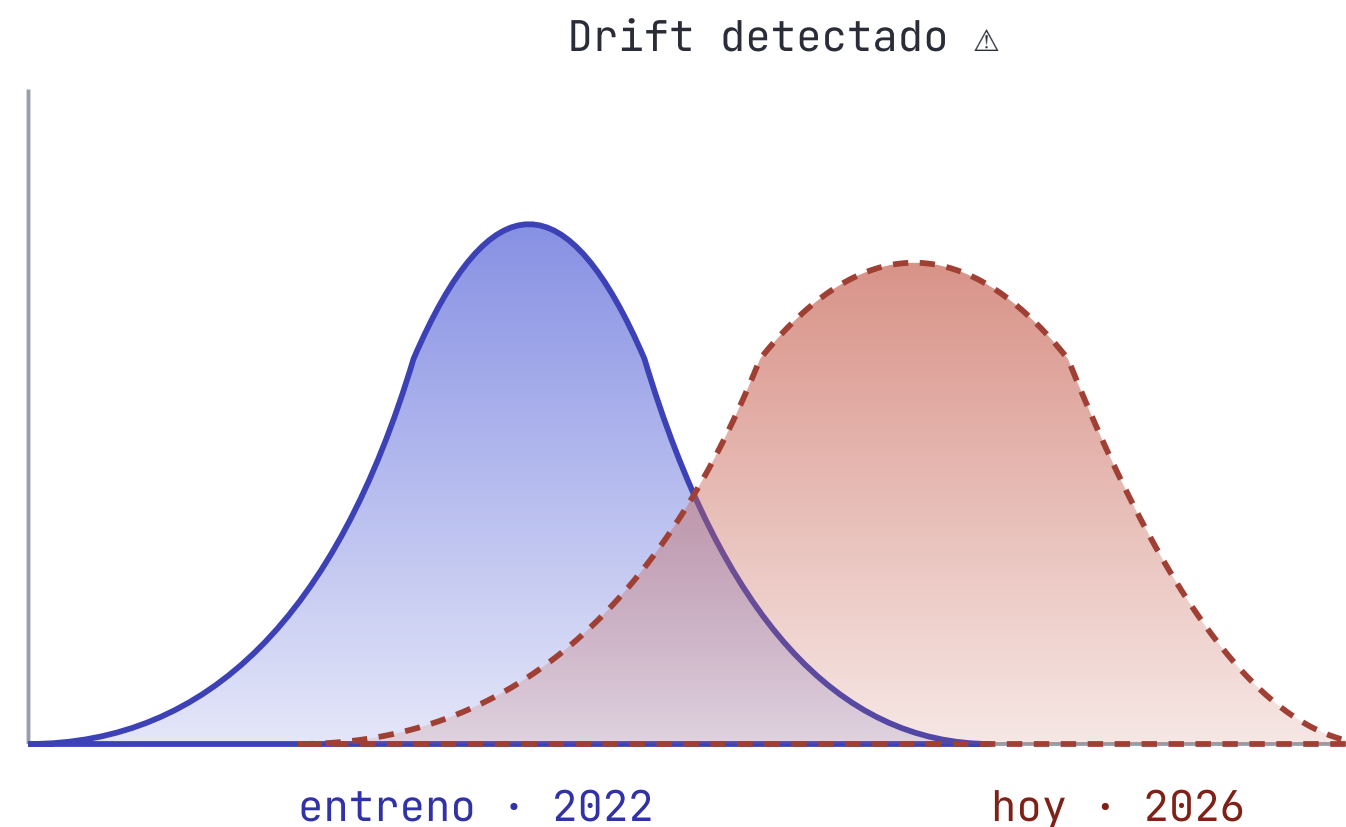
Data drift: cuando el mundo cambia, tu modelo no.

El modelo aprendió con datos del pasado. Si los datos de hoy son distintos, el modelo deja de funcionar bien — aunque su código no haya cambiado *una sola línea*.

- **Data drift**: cambian los datos de entrada (cambian las características de tus clientes)
- **Concept drift**: cambia la relación entre entrada y salida (lo que predices ya no significa lo mismo)
- **Causa típica**: estacionalidad, nuevos productos, cambios sociales, pandemia, regulaciones...

El caso clásico: modelos entrenados antes de marzo 2020 dejaron de funcionar de la noche a la mañana cuando la pandemia *reescribió el comportamiento humano*.

DISTRIBUCIÓN DE LA VARIABLE «GASTO MENSUAL»



Los datos de hoy ya no parecen los del entrenamiento. El modelo va a fallar.

El ecosistema *MLOps* en pocos nombres.

Hay decenas, pero estas son las que aparecen en cualquier proyecto serio. Las primeras dos son open-source y agnósticas. Las otras, ligadas a cada cloud.

MLFLOW

El estándar open-source

Track experimentos, gestiona modelos, registra versiones. Funciona en cualquier infraestructura.

Gratis, fácil de empezar, integrado con casi todo.

KUBEFLOW

MLOps nativo en Kubernetes

Pipelines de ML que corren en clústers K8s. Lo más usado en empresas que ya viven en Kubernetes.

Mayor curva de aprendizaje, mucho más potente a escala.

SAGEMAKER MODEL MONITOR

AWS · Drift y data quality

Detección automática de cambios en datos y predicciones. Alertas integradas con AWS CloudWatch.

Más caro pero «funciona de fábrica» en AWS.

VERTEX AI MODEL MONITORING

GCP · Drift y feature attribution

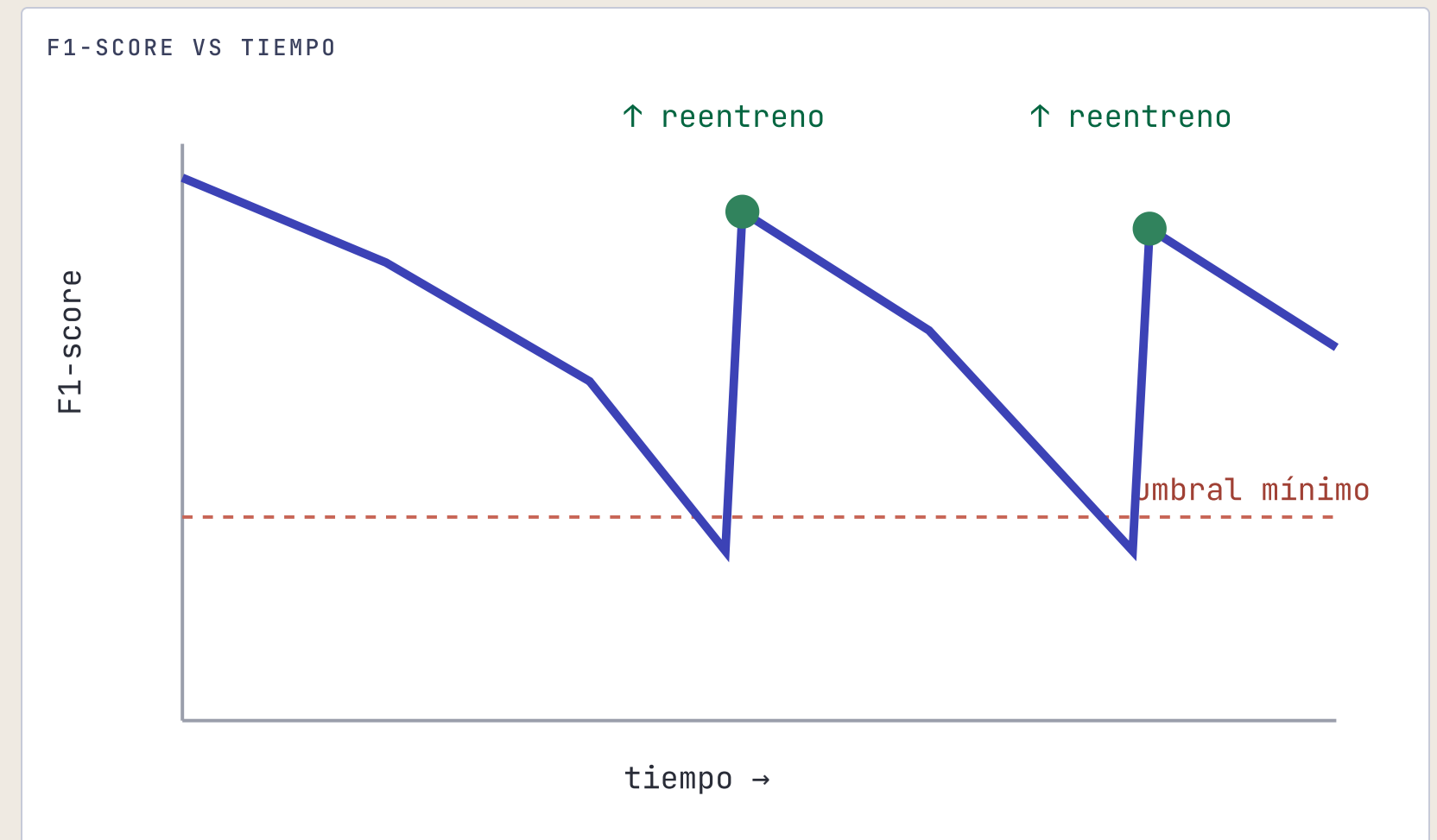
Monitoring continuo + explicabilidad. Alertas configurables. Equivalente al de AWS pero en GCP.

Bueno si ya tienes tu modelo en Vertex.

Reentrenar es *obligatorio*. Solo varía cuándo.

Es fundamental actualizar los modelos periódicamente porque pueden volverse obsoletos a medida que los patrones de datos cambian, lo que llevaría a predicciones inexactas o sesgadas.

- **Datos nuevos** disponibles que el modelo no ha visto
- Caída visible de *precisión, recall o F1*
- Drift detectado por el sistema de monitoreo
- Cambios en el negocio que cambian la respuesta correcta
- Decisión por calendario: «todos los lunes a las 3 AM»



Sin reentrenamiento, la curva cae por debajo del umbral. Reentrenar la sube otra vez.

Reentrenamiento *fijo* vs *dinámico*.

FIJO · POR CALENDARIO

Una vez por semana, mes o trimestre

Se reentrena cada cierto tiempo, llueva o truene. Decisión administrativa, no técnica.

- + Simple, predecible
- + Fácil de auditar y planificar
- – Reentrenas aunque no haga falta
- – Llegas tarde si el drift es brusco

Cuándo: sectores estables, regulaciones que exigen auditoría temporal.

DINÁMICO · POR TRIGGER

Cuando el sistema detecta degradación

El monitor dispara el pipeline de reentrenamiento al cruzar un umbral o detectar drift.

- + Reentrena solo cuando hace falta
- + Reacciona rápido a cambios bruscos
- – Más complejo de implementar
- – Requiere monitoreo confiable

Cuándo: entornos cambiantes, datos volátiles, equipos maduros.

En la práctica, los equipos maduros usan *híbridos*: trigger automático + revisión humana periódica.

Un día en la *vida* de un modelo desplegado.

08:00 El modelo de **predicción de churn** de GymPro lleva 3 meses en producción. Llamando cada noche a 200 socios «en riesgo» — y reteniendo al 35 %.

09:14 SageMaker Model Monitor lanza una alerta: *la distribución de la variable «edad»* ha cambiado. Llegan más socios de 18-25.

11:30 El equipo MLOps lo ve en el dashboard. *Confirman*: campaña en TikTok ha atraído un segmento que el modelo no vio en su entrenamiento.

12:00 Disparan el **pipeline de reentrenamiento** en Kubeflow. Toma los datos de los últimos 6 meses, vuelve a entrenar con el mismo código.

14:30 El nuevo modelo pasa los tests automáticos (CI). Se compara contra el actual con un 5 % de tráfico real (A/B). Mejora F1 en 4 puntos.

17:45 Promoción automática: el nuevo modelo va al 100 % del tráfico. *Rollback configurado* por si algo falla. Sin downtime.

22:00 El cron nocturno usa el modelo nuevo para llamar a los 200 socios. Nadie del equipo tuvo que tocar un servidor.

Esto es MLOps maduro: *detectar, reentrenar, validar, desplegar*. Ciclo cerrado. Sin pánico.

Cuatro señales de que *no estás listo*.

Desplegar antes de tiempo cuesta más caro que retrasarlo. Antes de soltar el modelo a producción, comprueba que **no estás en ninguna de estas cuatro**.

01

«Funciona en mi máquina»

Si no está en un contenedor reproducible, no está listo.

02

Cero monitorización

Si no puedes saber si el modelo se degrada, vas ciego. Inaceptable.

03

Sin plan de rollback

¿Qué pasa si el modelo nuevo va peor? Necesitas volver atrás en segundos, no en días.

04

Sin pipeline de reentrenamiento

Si reentrenar implica que alguien recuerde hacerlo a mano, no se hará. Y el modelo morirá.

De modelo entrenado a servicio vivo, en *siete* *pasos*.



Pasos 4-7 son un *bucle infinito*. Mientras el modelo viva en producción, nunca termina.

¿LISTOS PARA DESPLEGAR?

¿Qué *herramienta* usarías?

01 • Tu modelo de Python funciona en tu portátil pero falla en el servidor del cliente. *¿Primer paso?*

MIGRAR A KUBERNETES

EMPAQUETAR CON DOCKER

CAMBIAR A AWS

02 • Tu modelo de recomendaciones lleva 6 meses funcionando. De repente la *precisión cae un 12 %*. ¿Causa más probable?

BUG EN EL CÓDIGO

DATA DRIFT

SERVIDOR LENTO

03 • Modelo de fraude bancario. Los patrones cambian semana a semana. *¿Estrategia de reentrenamiento?*

NO HACE FALTA

FIJO CADA TRIMESTRE

DINÁMICO CON TRIGGERS

Cinco *ideas* que no olvidar.

01

Entrenar ≠ desplegar

El 87 % de los modelos nunca llega a producción. La diferencia está en el proceso.

02

MLOps = ML + DevOps

Automatizar todo: integración, despliegue, monitorización y reentrenamiento.

03

Docker empaqueta, K8s orquesta

Paella precocinada (Docker) servida desde la cocina industrial (Kubernetes).

04

Los modelos caducan

Data drift es la regla, no la excepción. Sin monitoreo + reentrenamiento, mueren.

05

Hamilton tenía razón

Diseña pensando en cómo fallará el sistema. *Antes* de desplegarlo, no después.

Modelos *en producción.*

Próxima sesión: casos reales, cierre del bloque de ML y proyecto final.

Preguntas, debate o café — en directo o por foro de la asignatura.